# Functions

Write a piece of code once. Give it inputs. Use its result.

Today we will write our own functions, use functions from libraries, and build a small Pokemon battle game.

# Part 1: Writing functions

From a named block of code to a result you can use.

1. **Quick recap**
   Lists, tuples and loops
2. **What is a function?**
   Code we can use again
3. **Defining and calling**
   Write it, then run it
4. **Parameters and arguments**
   Give a function inputs
5. **Return values**
   Get a result back
6. **Execution and scope**
   Follow calls and variable names

Practice: draw a face, track pizza slices, and trace a function.

# 01 - Quick recap

We can store several values and work through them with a loop.

```
scores = [12, 18, 24]          # A list
person = ("Mina", 20)          # A tuple

total = 0
for score in scores:
    total += score

print(total)                   # 54
print(person[0])               # Mina
```

Today, we will put code like this inside **functions**.

# 02 - What is a function?

A named piece of code that performs a task.

# You already call functions

```
name = input("Your name: ")
print(name)
print(len(name))
```

| Function | What it does |
| --- | --- |
| `input()` | Reads text and gives it back to us |
| `print()` | Displays something |
| `len()` | Gives us the length of something |

Some functions take inputs. Some give us a useful result.

# Why write our own?

Imagine using the same code in several places.

- Give that task a clear name.
- Call it whenever you need it.
- Change its code in one place instead of many.

```
show_menu()
```

A name such as `show_menu` tells us what the code is for. Next, we will learn how to write that function.

# 03 - Defining and calling

Defining writes the instructions. Calling runs them.

# Anatomy of a function

```
def show_menu():
    print("1. Play")
    print("2. Quit")
```

| Part | Meaning |
| --- | --- |
| `def` | Start a function definition |
| `show_menu` | The name we choose |
| `()` | Where parameters will go |
| `:` | The indented block starts next |
| Indented lines | The **body**: instructions indented by four spaces |

# A definition is not a call

```
def show_menu():
    print("1. Play")
    print("2. Quit")

print("Welcome!")
show_menu()
print("Choose an option.")
```

```
Welcome!
1. Play
2. Quit
Choose an option.
```

Python runs the body only when it reaches `show_menu()`.

# Call it again

```
def show_menu():
    print("1. Play")
    print("2. Quit")

show_menu()
show_menu()
```

Each call starts the body again.

Write the definition before the first call runs. Use parentheses to call a function: `show_menu()`, not just `show_menu`.

# 04 - Parameters and arguments

Run the same instructions with different inputs.

# Give a function an input

```
def show_score(score):
    print(f"Score: {score}")

show_score(12)
show_score(30)
```

```
Score: 12
Score: 30
```

**Parameter:** the name in the definition, such as `score`.

**Argument:** the value supplied by a call, such as `12`.

# More than one input

```
def show_score(name, score):
    print(f"{name}: {score}")

show_score("Mina", 12)
show_score("Sam", 30)
```

Arguments are matched to parameters in order.

You can also use their names:

```
show_score(score=12, name="Mina")
```

These are keyword arguments. The names must match the definition.

# A default value

```
def show_score(name, score=0):
    print(f"{name}: {score}")

show_score("Mina")       # Mina: 0
show_score("Sam", 30)    # Sam: 30
```

The default is used only when that argument is omitted.

Put parameters without defaults before parameters with defaults.

# Exercise: Draw a face

Write `draw_face(eyes, mouth)`.

- Print the eyes twice, with a space between them.
- Print the mouth on the next line, with one space before it.
- Call it twice with different characters. Choose your own!

For `draw_face("o", "-")`, the output is:

```
o o
 -
```

Folder: `exercises/07-functions/draw-a-face/`

# 05 - Return values

Give a result back to the code that called the function.

# A result we can use

```
def double(number):
    return number * 2

points = double(8)
print(points)               # 16
print(points + 5)           # 21
```

`double(8)` gives back `16`.

The code making the call is the **caller**. It can save the returned value, calculate with it, or use it in a condition.

# `print` AND `return` ARE DIFFERENT

## Display a value

```
def show_double(number):
    print(number * 2)

result = show_double(8)
print(result)
```

```
16
None
```

## Give back a value

```
def double(number):
    return number * 2

result = double(8)
print(result + 5)
```

```
21
```

Without an explicit `return`, a function returns `None`: no useful value.

# `return` ENDS THE CALL

```
def ticket_price(age):
    if age < 12:
        return 5
    return 10

print(ticket_price(8))       # 5
print(ticket_price(16))      # 10
```

Once Python reaches a `return`, it goes back to the caller.

For age `8`, the second `return` is never reached.

# Exercise: Pizza slices

Write `slices_left(slices, eaten)` that returns the slices remaining.

1. Start with 8 slices and eat 3.
2. Save the returned value in `remaining` and print it.
3. Use `remaining` in another call: eat 2 more slices.
4. Save and print the new result.

```
5
3
```

Assume no one eats more slices than are available.

Folder: `exercises/07-functions/pizza-slices/`

# 06 - How a function runs

Follow a call from start to finish.

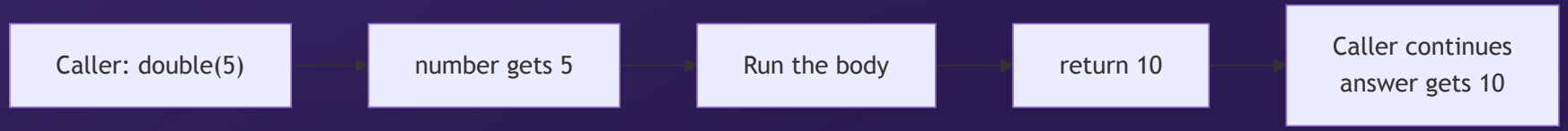
# Follow the value

```
def double(number):
    result = number * 2
    return result

answer = double(2 + 3)
print(answer)
```

1. Python defines `double`; it does not run the body yet.
2. It calculates the argument: `2 + 3` gives `5`.
3. It enters the function: the parameter `number` has value `5`.
4. It calculates `result`, then returns `10`.
5. The caller saves `10` in `answer` and prints it.

# The path of a call




The caller waits for the function's result before continuing.

If the body reaches its end without a `return`, the result is `None`.

# One function can call another

```
def double(number):
    return number * 2

def double_plus_one(number):
    result = double(number)
    return result + 1

answer = double_plus_one(3)
print(answer)               # 7
```

`double_plus_one(3)` calls `double(3)` inside its body.

`double(3)` returns `6`. Then `double_plus_one` adds `1` and returns `7`. Both functions can also be called on their own.

# 07 - Variable scope

Scope means where a variable name can be used.

# Parameters and variables are local

```
def double(number):
    result = number * 2
    return result

answer = double(4)
print(answer)               # 8
```

`number` and `result` are local: their names belong to this function call.

`answer` belongs to the code outside the function. Returning `8` does not make the name `result` available outside.

# A local name is not available outside

```
def double(number):
    result = number * 2
    return result

answer = double(4)
print(result)                # NameError
```

The caller received the **value**, not the function's local variable name.

Use `answer` here, not `result`.

# Same name, different variables

```
score = 10                  # A global name: outside functions

def show_score():
    score = 3               # A local name: inside this call
    print(score)

show_score()                # 3
print(score)                # 10
```

Assignment inside the function normally creates a local variable. It does not reassign the global `score`.

# Each call has its own local names

```
def first_point():
    points = 0
    points += 1
    return points

print(first_point())        # 1
print(first_point())        # 1
```

The second call starts again at `points = 0`.

Local variables are not shared between separate calls.

# Reading a global name

```
school = "ReDI"

def show_school():
    print(school)

show_school()                # ReDI
```

A function can read a name defined outside it.

Usually, prefer an explicit input: `show_school(school)`. Parameters and return values make a function's inputs and outputs easier to see.

# Trace execution and scope

```
value = 10

def add_two(value):
    result = value + 2
    print(result)
    return result

answer = add_two(1)
print(answer)
print(value)
```

What three lines are printed? Could we use `result` outside the function?

Task and answers are in the lesson’s exercise and solution READMEs.

# Break

15 minutes

# Part 2: Reusing code

Document our functions, use libraries, and build a game.

8 **Docstrings**
Explain a function's purpose

9 **Libraries and imports**
Use code supplied by others

10 **Installing with pip**
Fetch real Pokemon data

11 **Mini project**
A simple Pokemon battle

12 **Homework and questions**
Practise the main ideas

# 08 - Docstrings

A **docstring** is a short description inside a function. It is the first statement in the body.

```
def double(number):
    """Return twice the supplied number."""
    return number * 2
```

Use triple quotes. Describe what the function does.

For longer functions, also explain their inputs and returned result.

# Read a function's help

```
def double(number):
    """Return twice the supplied number."""
    return number * 2

help(double)
```

Python shows the function's name, parameters, and docstring.

Notice `help(double)`, **not** `help(double(4))`. We want help about the function, not about its returned number.

# Comments and docstrings

```
def circle_area(radius):
    """Return the area of a circle with this radius."""
    # The calculation goes here.
    pass
```

- A **docstring** tells callers what the function is for.
- A **comment** can explain a particular line of code.
- `pass` is a placeholder that does nothing. Replace it with working code.

You will add docstrings to both functions in the battle project.

# 09 - Libraries and imports

We do not have to write everything ourselves.

# What is a library?

A **library** is reusable code supplied for other programs to use.

A **module** is a unit of Python code we can import, often a `.py` file. A library can contain several modules.

Python includes a **standard library**. It is installed with Python.

| Module | Useful for |
| --- | --- |
| `math` | Mathematical functions and constants |
| `random` | Random choices and numbers |

# Import a module

```
import math

print(math.sqrt(25))        # 5.0
print(round(math.pi, 2))    # 3.14
```

`import math` makes the module available in this program.

The dot means "use this name from that module": `math.sqrt` is the square-root function in `math`.

# Import a function by name

```
from math import sqrt

print(sqrt(25))             # 5.0
```

Now we can call `sqrt()` without writing `math.`.

Import only the names you need. Avoid `from math import *`, which makes it harder to see where names came from.

# An import can have a shorter name

```
import random as rand

print(rand.randint(1, 6))
```

`as` gives an imported name an **alias**: another name to use here.

It does not install or change the library. Use clear names; an alias is optional.

# Random choices and numbers

```python
import random

names = ["pikachu", "bulbasaur", "charmander"]
opponent = random.choice(names)
damage = random.randint(5, 15)

print(opponent)
print(damage)
```

`choice()` picks one item from a non-empty sequence.

`randint(5, 15)` gives an integer from **5 through 15**, including both ends.

The output can change each time we run the program.

# Exercise: Circle area

Write `circle_area(radius)` that **returns**:

```
math.pi * radius * radius
```

Import `math`. Call your function with radius `2`. Print its result rounded to two decimal places.

```
12.57
```

Keep the unrounded value inside the function.

Folder: `exercises/07-functions/circle-area/`

# 10 - Installing with pip

Use a library that does not come with Python.

# Install first, then import

**pip** installs Python packages. A **package** is an installable bundle of code and other files.

We will use `pokebase` to fetch real Pokemon data.

- Install it once in your Python environment.
- Import it in each program that uses it.
- Installation and fetching the data need internet access.

Installing a package does **not** run it in your program. Importing a missing package causes `ModuleNotFoundError`.

# Give the project its own packages

A **virtual environment** is a folder containing a separate Python environment. We will name the folder `.venv`.

Open your exercise folder in VS Code and choose **Terminal → New Terminal**.

## Windows (PowerShell)

```
py -m venv .venv
```

## macOS

```
python3 -m venv .venv
```

Do this once per exercise folder that needs the package.

# Install using that environment

## Windows (PowerShell)

```
.\.venv\Scripts\python.exe -m pip install pokebase==1.4.1
```

Then run your code:

```
.\.venv\Scripts\python.exe exercise.py
```

## macOS

```
.venv/bin/python -m pip install pokebase==1.4.1
```

Then run your code:

```
.venv/bin/python exercise.py
```

`-m pip` runs pip using that Python. No activation command is needed. Do not commit `.venv`; it is not part of your solution.

# Fetch data with a library function

```
import pokebase

pokemon = pokebase.pokemon("pikachu")
stats = []
for stat in pokemon.stats:
    stats.append(stat.base_stat)

print(stats)                # [35, 55, 40, 50, 50, 90]
```

The library handles downloading and reading the data for us.

`pokemon.stats` gives us a list of stats. Each `stat.base_stat` is a number: health, attack, defense, special attack, special defense, and speed.

We can loop over that list. **No dictionaries are needed.**

# External data can fail

- Check spelling if a name is not found. Try `pikachu` first.
- Check your connection if the data cannot be downloaded.
- Use the same Python environment to install and run the program.

The project starter includes the error-handling code. You do not need to write it yourself.

The checkers use sample data: they work without internet or installed packages. Run the program with `pokebase` to try the real data.

# Exercise: Pokemon stats

Fetch Pikachu using `pokebase`.

- Loop over `pokemon.stats`; each number is in `stat.base_stat`.
- Use a loop to add them together. Do not use `sum()`.
- Print the total.

```
320
```

The numbers are external data; the calculation is our own Python code.

Folder: `exercises/07-functions/pokemon-stats/`

# 11 - Mini project: Pokemon battle

Your functions make the game work.

# The rules

HP means health points.

- Enter any supported Pokemon name.
- The computer chooses its Pokemon from a short list.
- Both start with their real health from the library.
- Press Enter to attack. Each hit does random damage from 5 to 15.
- If the opponent survives, it attacks back.
- The first Pokemon to reach zero health loses.

This is a simplified classroom game, not the official Pokemon battle system.

Folder: `exercises/07-functions/mini-project-pokemon-battle/`

# Write both functions

The starter has placeholders, **not function solutions**.

| Function | What students implement |
| --- | --- |
| `get_hp(name)` | Fetch the Pokemon and return `pokemon.stats[0].base_stat`: its health |
| `attack(hp, damage)` | Subtract damage and return the remaining health, never below zero |

Add a docstring to each function.

The input and error-handling code is already supplied. It asks again for unknown names and stops clearly if the service fails.

# Connect your functions to the loop

The starter supplies this structure. Fill the two missing function calls.

```
while player_hp > 0 and opponent_hp > 0:
    input("Press Enter to attack: ")
    damage = random.randint(5, 15)
    # Update opponent_hp using attack(...).
    print(f"{opponent_name} has {opponent_hp} HP.")

    if opponent_hp > 0:
        damage = random.randint(5, 15)
        # Update player_hp using attack(...).
        print(f"{player_name} has {player_hp} HP.")
```

Save each returned result. Do not let a defeated opponent attack back.

# Try your game

```
Your Pokemon: pikachu
You: pikachu (35 HP)
Opponent: bulbasaur (45 HP)
Press Enter to attack:
bulbasaur has 33 HP.
pikachu has 28 HP.
```

This is one possible start. The opponent and damage can change.

Check that health never goes below zero and the game ends. Run `check.py` for predictable tests using sample data.

# 12 - Homework

Four short tasks. Each has an `exercise.py` and its own `check.py`.

| Task | Practise |
|---|---|
| 1. Ticket prices | Parameters, conditions and returning a price |
| 2. High scores | A list parameter and a loop |
| 3. Average score | Returning a calculation for the caller to use |
| 4. Roll a die | Default parameters, a library and a docstring |

Folder: `exercises/07-functions/`

# Homework 1: Ticket prices

Write `ticket_price(age)`.

- Return `5` if the age is below `12`.
- Otherwise return `10`.
- Print the returned prices for ages `8` and `16`.
- Add those two returned prices and print the total.

```
5
10
15
```

Folder: `homework-01-ticket-prices/`

# Homework 2: High scores

Write `count_high_scores(scores, minimum)`.

Use a loop to count scores **greater than or equal to** `minimum`. Return the count.

Use `[12, 45, 7, 30]` and a minimum of `20`. Print the result.

```
2
```

Folder: `homework-02-high-scores/`

# Homework 3: Average score

Write `average(scores)`. Assume the list is non-empty.

Use a loop to find the total, then return `total / len(scores)`.

Call it with `[12, 18, 24]`. Print the result. If it is at least `15`, also print `Pass`.

```
18.0
Pass
```

Folder: `homework-03-average-score/`

# Homework 4: Roll a die

Import `random`. Write `roll_die(sides=6)` with a docstring.

Return a random integer from `1` through `sides`, including both ends.

Print one roll using the default and one using `sides=20`.

One possible output:

```
4
17
```

Folder: `homework-04-roll-a-die/`

# The main ideas

- **Define** a function with `def`; **call** it with parentheses.
- **Parameters** name the inputs; **arguments** supply their values.
- **Return** gives a result to the caller; **print** displays something.
- **Scope** determines where a variable name is available.
- **Docstrings** describe what a function does.
- **Import** reusable code; **pip** installs additional packages.

Use small functions with clear inputs and useful results.

# Questions?